

Exam exercise for Module 4: A model for a DC-motor



The purpose of this exam question is to provide a model for how the velocity of a DC-motor depends on the input voltage. For this, data was collected from an experiment. Input-output data are recorded in the data set below. The first column is the input voltage (V) and the second column is the motor velocity (rad/s). The sampling time is 0.01 s. A linear regression with ARMA noise should be obtained.

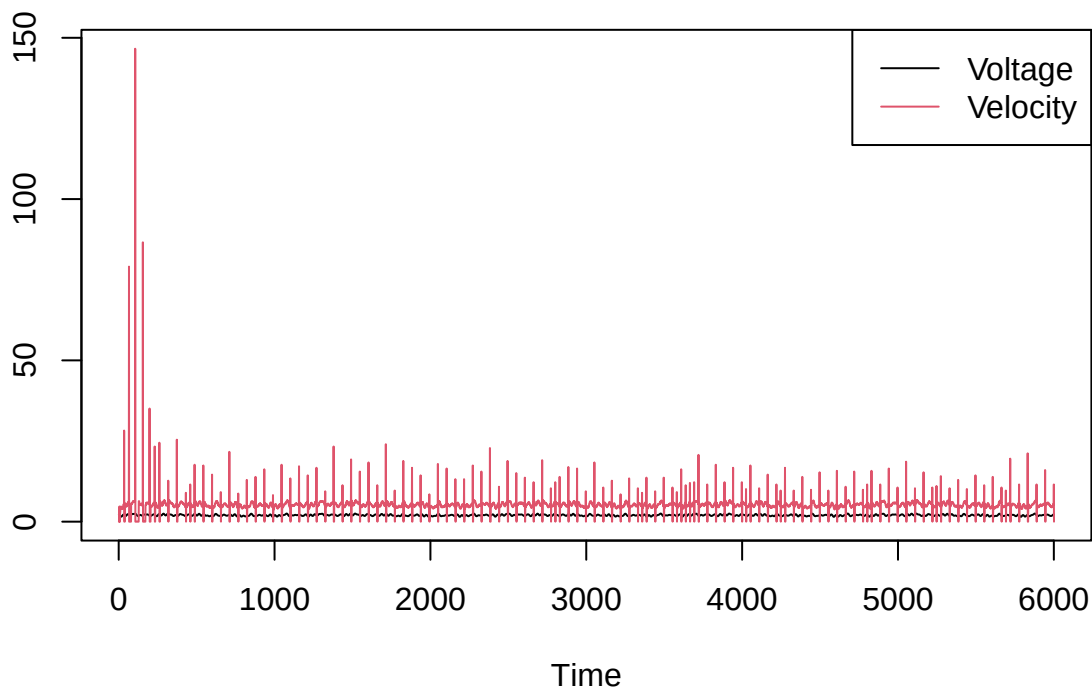
Viewing and cleaning data

- Load the data into R:

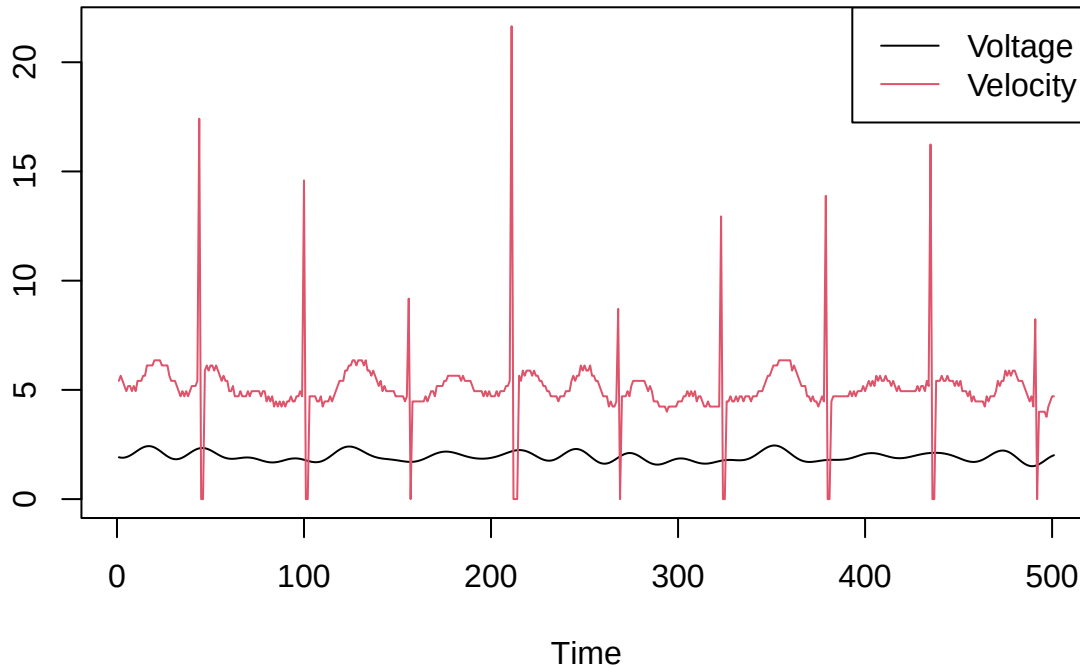
```
dat = as.ts(read.table("https://asta.math.aau.dk/datasets?file=dcmotor.txt"),
             colnames=c("Voltage", "Velocity"))
```

1. Start by plotting the data (both input and output). You may want to zoom in on a part to get a better view.

```
ts.plot(dat,col=1:2); legend("topright",legend=c("Voltage", "Velocity"),col=1:2,lty=1)
```

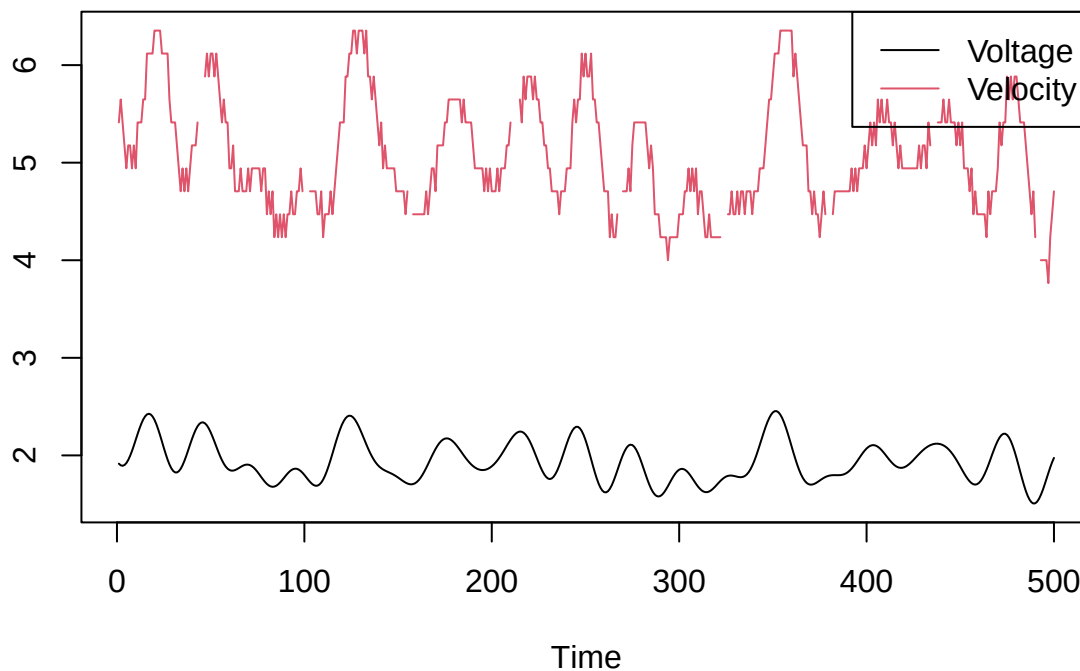


```
ts.plot(dat[500:1000,],col=1:2); legend("topright",legend=c("Voltage","Velocity"),col=1:2,lty=1)
```



2. It should be obvious that the output data is subject to noise (especially in the beginning) and a regularly repeating unknown disturbance. Choose a part of the data that sounds reasonable for obtaining the model. For example, here is a part of the data with the first 500 measurements removed, and all measurements outside the interval $[3, 7]$ removed.

```
datX = dat[500:6001,] # removing start
datX[datX[,2]<3|datX[,2]>7,2]=NA # exchanging velocity outliers outside interval [3,7] with NA
ts.plot(datX[1:500,],col=1:2); legend("topright",legend=c("Voltage","Velocity"),col=1:2,lty=1)
```



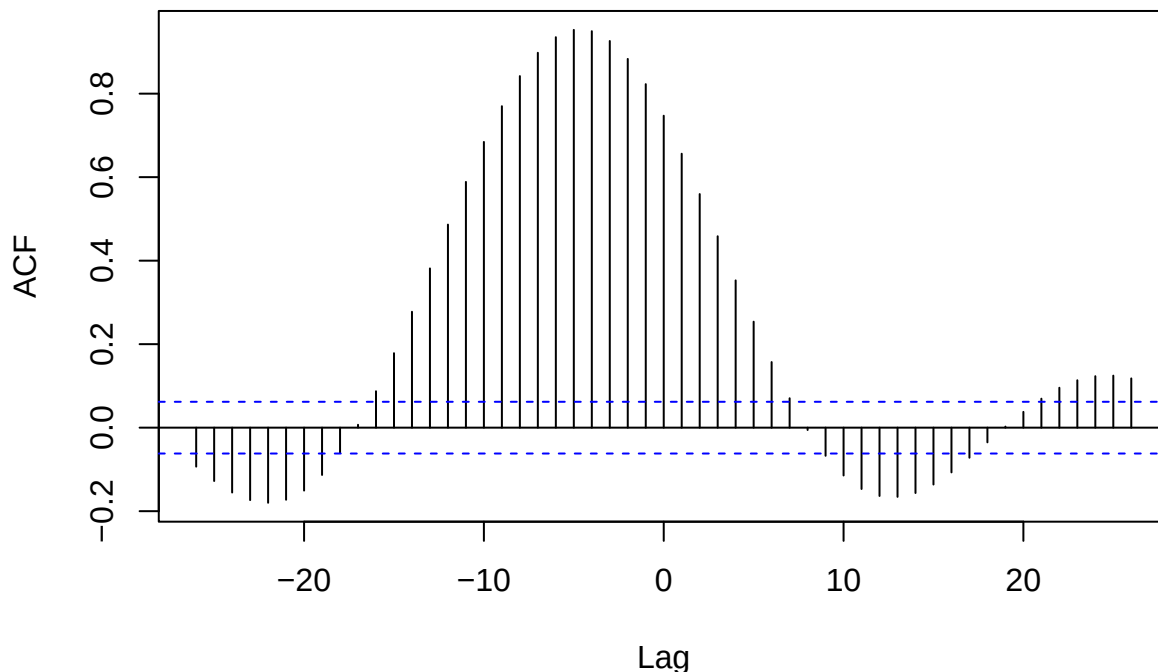
```
dat1 = datX[1000:2000,] # cherry-picking a time interval
```

A linear regression model with ARMA noise

3. Use the cross-correlation function to detect the delay between the input and output (NA values can be handled in the ccf as well as the acf by using the option na.action=na.pass).
 - Explain the cross-correlation function.

```
cc = ccf(dat1[, "Voltage"], dat1[, "Velocity"], na.action=na.pass)
```

dat1[, "Voltage"] & dat1[, "Velocity"]



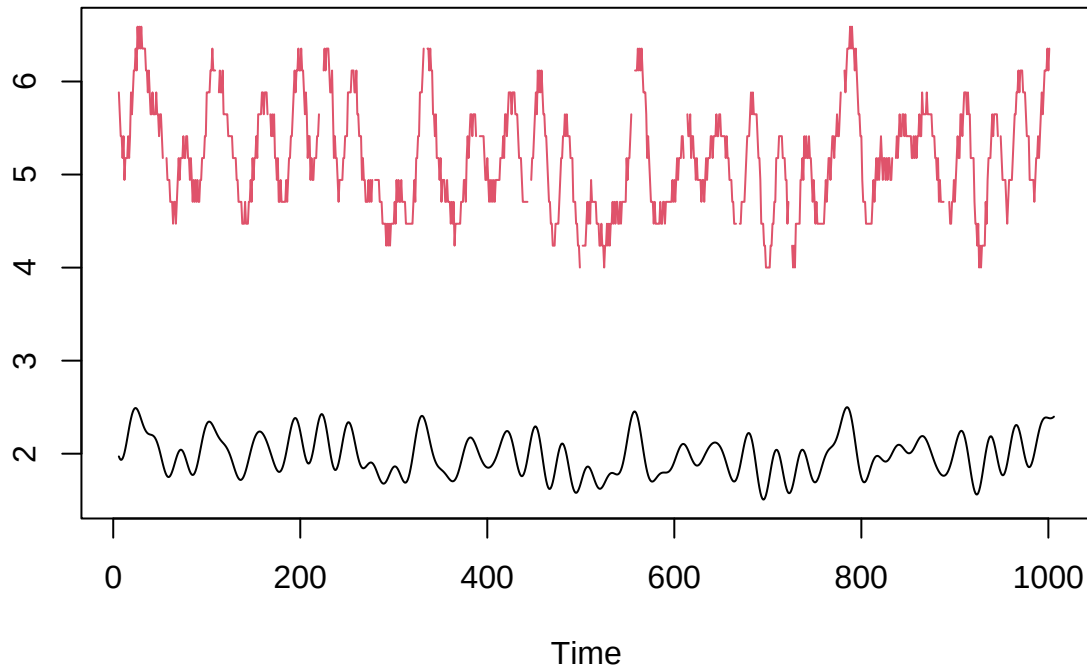
```
estlag = cc$lag[which(cc$acf==max(abs(cc$acf)))]
estlag # in samples, but since sampling time is 0.01s, 1 sample = 10ms
```

```
## [1] -5
```

The cross-correlation function describes the linear dependence between two time series as a function of a time lag. Peaks in the CCF indicate delays where the input has the strongest influence on the output. The lag corresponding to the maximum absolute correlation provides an estimate of the system delay, which is required when formulating the regression model. There is a system delay of -40ms.

4. Fit a linear regression model with AR(1) noise for the output using the input as exogenous variable (adjust for the delay using the lag() function in R). What is the autocorrelation function for the noise process in this model?

```
dat_shifted = ts.intersect(lag(as.ts(dat1[, "Voltage"]),estlag),dat1[, "Velocity" ])
colnames(dat_shifted) <- c("Voltage", "Velocity")
ts.plot(dat_shifted[, "Voltage"],dat_shifted[, "Velocity"],col=1:2)
```



```
mod=arima(dat_shifted[, "Velocity"],order=c(1,0,0),xreg=dat_shifted[, "Voltage"]); mod
```

```
##
## Call:
## arima(x = dat_shifted[, "Velocity"], order = c(1, 0, 0), xreg = dat_shifted[,
##      "Voltage"])
##
## Coefficients:
##          ar1  intercept  dat_shifted[, "Voltage"]
##      0.8584      1.6348          1.7945
## s.e.  0.0168      0.2896          0.1442
##
## sigma^2 estimated as 0.0355:  log likelihood = 227.55,  aic = -447.1
```

This results in the model:

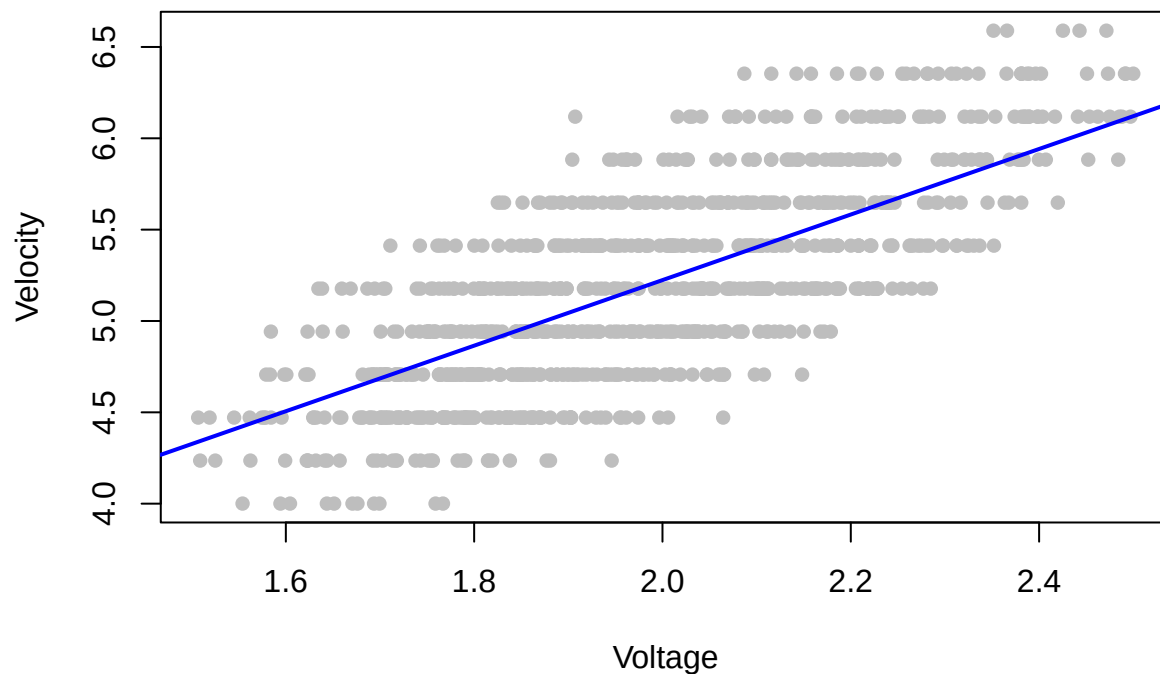
$$\omega_t = \beta_0 + \beta_1 \cdot V_t + \alpha \cdot \epsilon_{t-1} + W_t = 1.6348 + 1.7945 \cdot V_t + 0.8584 \cdot \epsilon_{t-1} + W_t$$

The autocorrelation function measures how correlated X_t and X_{t+h} are related. Assuming an AR(1) and stationary process (each value depends on previous value + noise, and the mean and variance are constant):

$$\rho(t, t+h) = \alpha^h = 0.8584^h$$

Plotting the model:

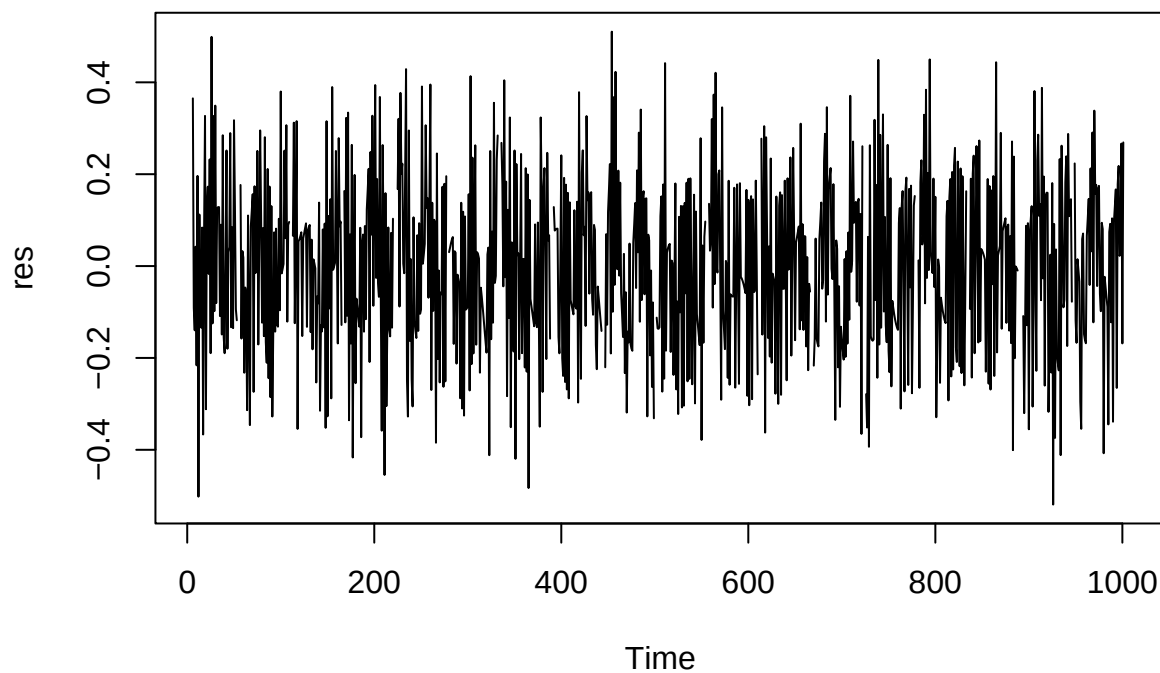
```
# Scatter plot of Voltage vs Velocity
plot(dat_shifted[, "Voltage"], dat_shifted[, "Velocity"], pch = 16, col = "grey", xlab = "Voltage", ylab = "Velocity")
abline(a = 1.6348, b = 1.7945, col = "blue", lwd = 2)
```



5.

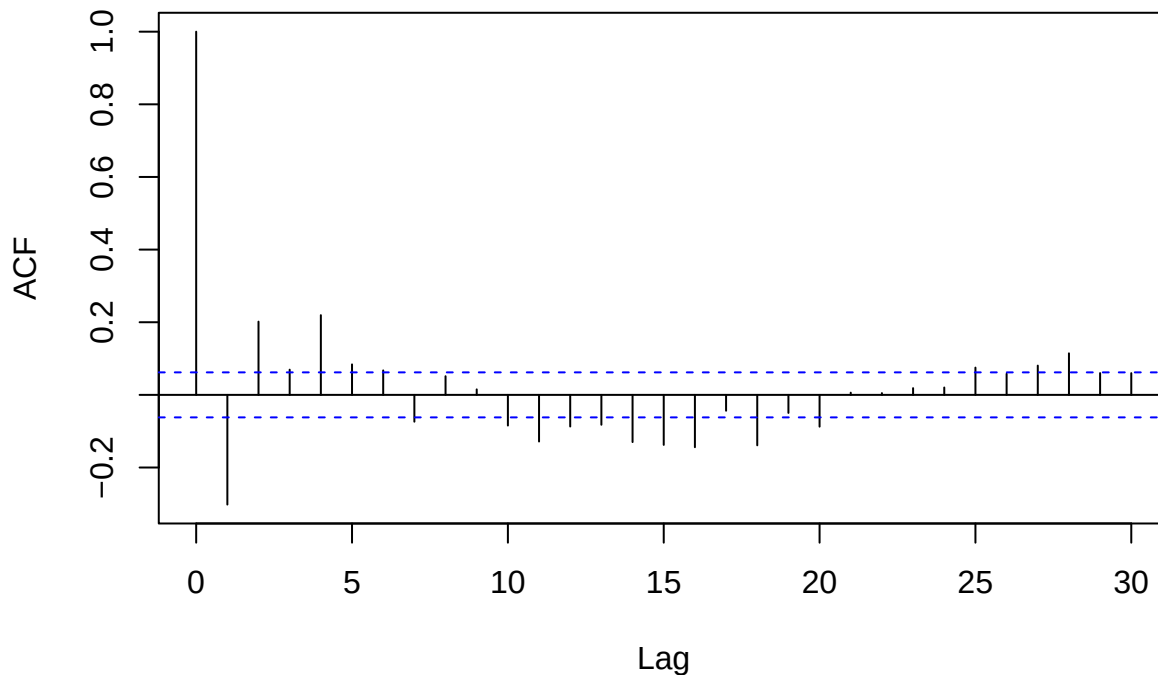
Check whether the AR(1) model fits the data by considering the residuals. - Use a time series plot of the residuals as well as a correlogram. - Explain what you expect to see if your model is correct.

```
res=resid(mod)
plot(res)
```



```
acf(res, na.action=na.pass)
```

Series res



The residuals should look like white noise in both plots. As expected, The time series looks like a random distribution with $\mu = 0$ and the correlogram fits the expected shape for white noise, although there remains some of the oscillations which are meaningful from the original data.

6. Fit a linear regression model with ARMA(p,q) noise for different choices of p and q. Compare the models using AIC and pick the preferred model (if you get a warning message, try using the option `optim.control=list(maxit = 1000)`).

```
# List of ARMA(p,q) orders you want to try
# All combinations of p and q from 0 to 3
p_values <- 0:3
q_values <- 0:3

# Data frame to store results
results <- data.frame(p=integer(),
                      q=integer(),
                      AIC=numeric(),
                      stringsAsFactors=FALSE)

# Loop over all combinations
for (p in p_values) {
  for (q in q_values) {
    fit <- try(arima(dat_shifted[, "Velocity"],
                    order = c(p,0,q),
                    xreg = dat_shifted[, "Voltage"],
                    optim.control = list(maxit=1000)),
              silent = TRUE)

    if (!inherits(fit, "try-error")) {
      cat("Fitted ARMA(", p, ", 0, ", q, ")\\n")
    }
  }
}
```

```

    results <- rbind(results, data.frame(p=p, q=q, AIC=fit$aic))
  } else {
    cat("ARMA(", p, ",0,", q, ") failed to converge\n")
  }
}
}

```

```

## Fitted ARMA( 0 ,0, 0 )
## Fitted ARMA( 0 ,0, 1 )
## Fitted ARMA( 0 ,0, 2 )
## Fitted ARMA( 0 ,0, 3 )
## Fitted ARMA( 1 ,0, 0 )
## Fitted ARMA( 1 ,0, 1 )
## Fitted ARMA( 1 ,0, 2 )
## Fitted ARMA( 1 ,0, 3 )
## Fitted ARMA( 2 ,0, 0 )
## Fitted ARMA( 2 ,0, 1 )
## Fitted ARMA( 2 ,0, 2 )

```

```

## Warning in arima(dat_shifted[, "Velocity"], order = c(p, 0, q), xreg =
## dat_shifted[, : possible convergence problem: optim gave code = 1

```

```

## Fitted ARMA( 2 ,0, 3 )
## Fitted ARMA( 3 ,0, 0 )
## Fitted ARMA( 3 ,0, 1 )
## Fitted ARMA( 3 ,0, 2 )
## Fitted ARMA( 3 ,0, 3 )

```

```

print(results)

```

```

##      p q      AIC
## 1  0 0  780.72437
## 2  0 1  247.31250
## 3  0 2 -78.74443
## 4  0 3 -208.93120
## 5  1 0 -447.09979
## 6  1 1 -537.02235
## 7  1 2 -596.81330
## 8  1 3 -713.84759
## 9  2 0 -569.87636
## 10 2 1 -568.38824
## 11 2 2 -565.91753
## 12 2 3 -970.77009
## 13 3 0 -568.69875
## 14 3 1 -567.06614
## 15 3 2 -564.39158
## 16 3 3 -980.50312

```

```

# Select the best model

```

```

best_model <- results[which.min(results$AIC), ]
cat("Best model is ARMA(", best_model$p, ",0,", best_model$q, ") with AIC =", best_model$AIC, "\n")

```

```

## Best model is ARMA( 3 ,0, 3 ) with AIC = -980.5031

```

Apparently, the higher the order, the lower the AIC. We choose the maximum suggested value for p and q of 3.

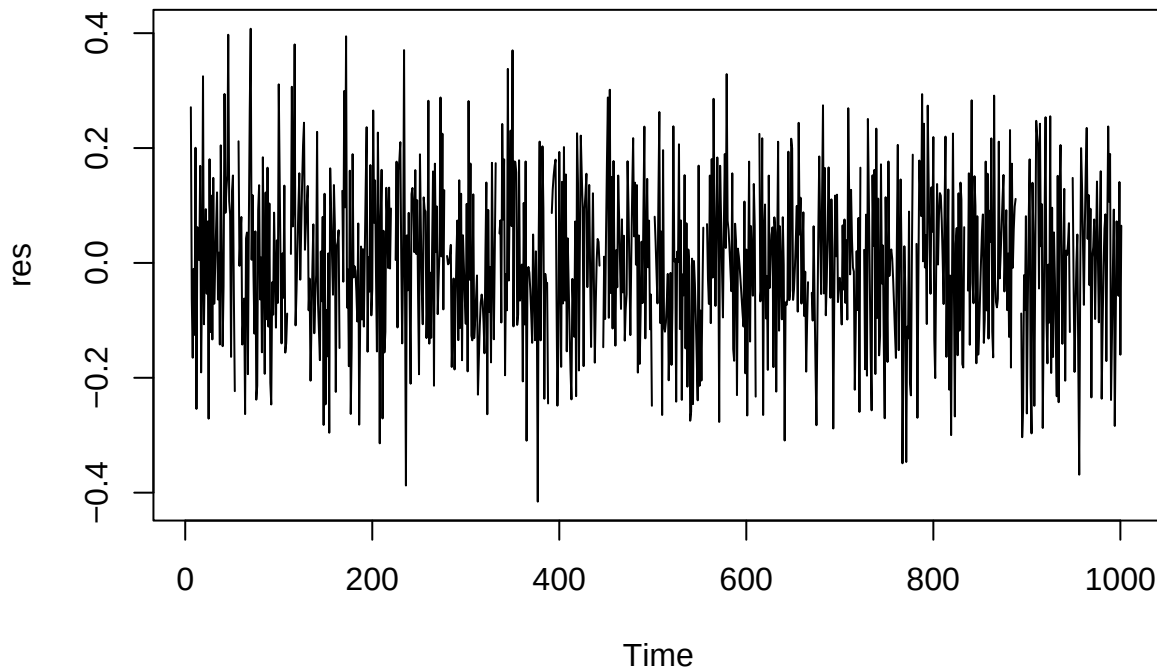
7. Check whether the chosen model fits the data by considering the correlogram for the residuals.

- If the model does not fit well you may consider returning to the previous step and increase the order of the ARMA(p,q) model up to $p \leq 3$ and $q \leq 3$ to see if you can get a model that fits better.

```
mod=arima(dat_shifted[, "Velocity"],order=c(3,0,3),xreg=dat_shifted[, "Voltage"], optim.control = list(maxit=1000))
mod
```

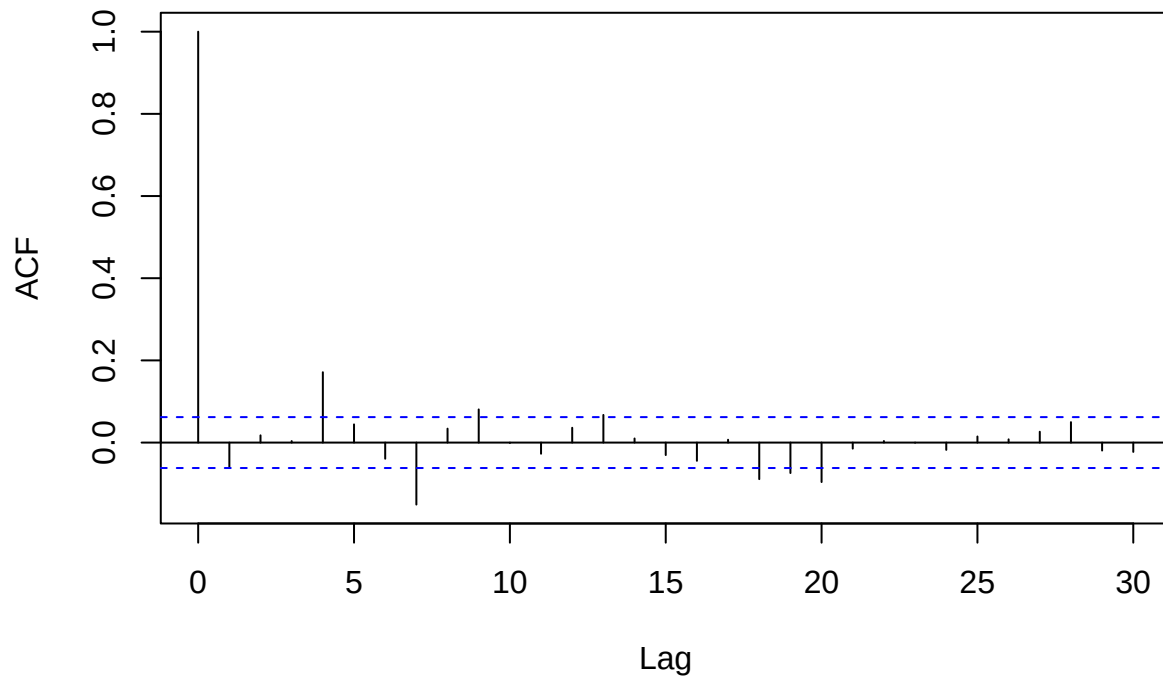
```
##
## Call:
## arima(x = dat_shifted[, "Velocity"], order = c(3, 0, 3), xreg = dat_shifted[,
##      "Voltage"], optim.control = list(maxit = 1000))
##
## Coefficients:
## Warning in sqrt(diag(x$var.coef)): NaNs produced
##      ar1      ar2      ar3      ma1      ma2      ma3  intercept
##      0.9241  0.955 -0.9682 -0.7027 -0.8287  0.8740    0.4916
## s.e.  0.0088   NaN   0.0088  0.0217  0.0097  0.0169    0.2225
##      dat_shifted[, "Voltage"]
##                      2.3687
## s.e.                  0.1118
##
## sigma^2 estimated as 0.0199:  log likelihood = 499.25,  aic = -980.5
```

```
res=resid(mod)
plot(res)
```



```
acf(res, na.action=na.pass)
```

Series res



8. When you have a model that fits the data, write out the model and interpret the estimated parameters.

The selected model is a linear regression with ARMA(3,3) errors:

$$\omega_t = \beta_0 + \beta_1 \cdot V_t + n_t, n_t = \sum_{i=1}^3 \phi_i n_{t-i} + \sum_{j=1}^3 \theta_j \varepsilon_{t-j} + \varepsilon_t$$

$$\omega_t = 0.4916 + 2.3687 V_t + n_t,$$

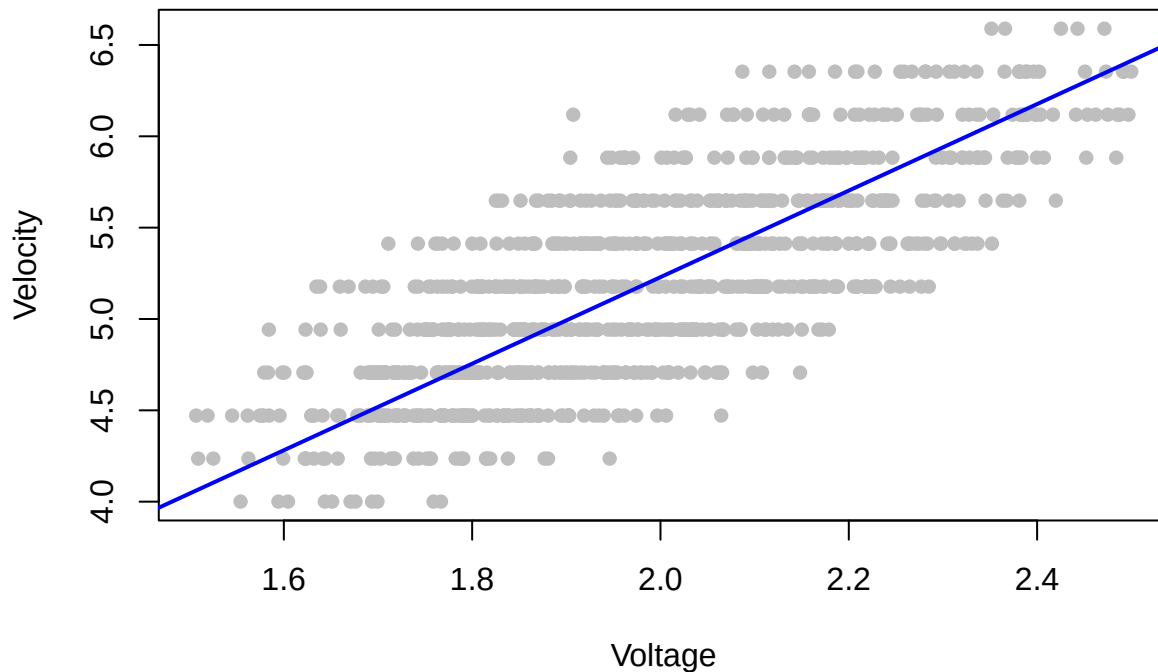
$$n_t = 0.9241 n_{t-1} + 0.9550 n_{t-2} - 0.9682 n_{t-3}$$

$$+ \varepsilon_t - 0.7027 \varepsilon_{t-1} - 0.8287 \varepsilon_{t-2} + 0.8740 \varepsilon_{t-3}$$

```
# Scatter plot of Voltage vs Velocity
```

```
plot(dat_shifted[, "Voltage"], dat_shifted[, "Velocity"], pch = 16, col = "grey", xlab = "Voltage", ylab = "Velocity")
```

```
abline(a = 0.4916, b = 2.3687, col = "blue", lwd = 2)
```



9. Make a confidence interval for the slope of the regression.

```
slope <- mod$coef["dat_shifted[, \"Voltage\"]"]

# Extract the standard error
se_slope <- sqrt(mod$var.coef["dat_shifted[, \"Voltage\"]", "dat_shifted[, \"Voltage\"]"])

# Calculate 95% Confidence Interval
lower_ci <- slope - 2 * se_slope
upper_ci <- slope + 2 * se_slope

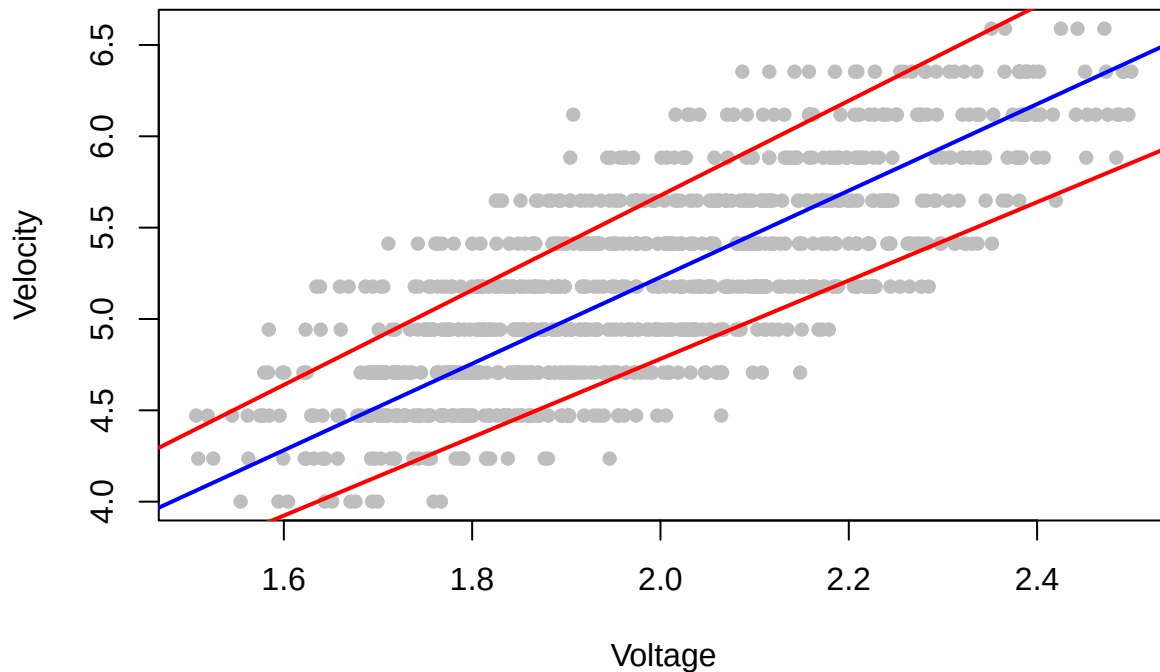
cat("Slope Estimate:", slope, "\n")

## Slope Estimate: 2.368654

cat("95% CI: [", lower_ci, ",", upper_ci, "]\n")

## 95% CI: [ 2.145045 , 2.592264 ]

plot(dat_shifted[, "Voltage"], dat_shifted[, "Velocity"], pch = 16, col = "grey", xlab = "Voltage", ylab = "Velocity")
abline(a = 0.4916, b = 2.3687, col = "blue", lwd = 2)
abline(a = 0.4916, b = lower_ci, col = "red", lwd = 2)
abline(a = 0.4916, b = upper_ci, col = "red", lwd = 2)
```



10.

Suppose the next value of the voltage is 2.23. Predict the next observation of velocity and make a confidence interval for the prediction.

```
# New data for prediction
new_voltage <- 2.23
new_xreg <- matrix(new_voltage, nrow = 1)

# Predict the next value using the fitted model
pred <- predict(mod, n.ahead = 1, newxreg = new_xreg)
pred

## $pred
## Time Series:
## Start = 1007
## End = 1007
## Frequency = 1
## [1] 5.85348
##
## $se
## Time Series:
## Start = 1007
## End = 1007
## Frequency = 1
## [1] 0.1571207

predicted_velocity <- pred$pred

cat("Predicted velocity at ", new_voltage, " V:", predicted_velocity, " rad/s\n")

## Predicted velocity at  2.23  V: 5.85348  rad/s

se_pred <- pred$se

# Calculate the 95% prediction interval
lower_pred_ci <- predicted_velocity - 2 * se_pred
```

```
upper_pred_ci <- predicted_velocity + 2 * se_pred
cat("95% Prediction Interval: [", lower_pred_ci, ",", upper_pred_ci, "]\n")

## 95% Prediction Interval: [ 5.539239 , 6.167722 ]
```